

Clase 330 — Análisis de código y automatización de seguridad

Parte: **17 — Profundización para certificaciones** · Fuente: *CompTIA PenTest+ — Tools and Code Analysis* · (ISC)² *CISSP OSG — Software Development Security* · *OWASP Code Review Guide* / *ASVS* 🕒
Duración estimada: **150 min** · Nivel: **Avanzado**

Objetivo

Encontrar y **corregir** vulnerabilidades en el código antes de que lleguen a producción, y hacerlo de forma **automatizada y repetible**. Esta clase cubre la **revisión de código segura** (manual, guiada por OWASP), las cuatro familias de análisis —**SAST, DAST, IAST y SCA**—, su **integración en el pipeline de CI**, el **triaje de hallazgos** (severidad, falsos positivos, deuda) y el **scripting de automatización de seguridad** para pegar las piezas. El enfoque es **defensivo**: se trata de detectar y remediar, no de explotar. Cierra *Software Development Security* de CISSP y *Tools and Code Analysis* de PenTest+.

Resultados de aprendizaje

Al finalizar, el alumno podrá:

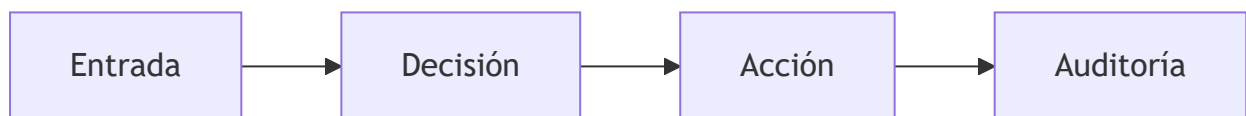
1. **Realizar** una revisión de código segura guiada por checklist (OWASP Code Review / ASVS) sobre un fragmento vulnerable.
2. **Distinguir** SAST, DAST, IAST y SCA por lo que ven, cuándo se ejecutan y qué falsos positivos/negativos aportan.
3. **Integrar** SAST y SCA en un pipeline de CI (GitHub Actions) que falle ante hallazgos por encima de un umbral.
4. **Triar** hallazgos por severidad y explotabilidad, separando verdaderos positivos de ruido y gestionando excepciones.
5. **Escribir** un script de automatización que ejecute análisis, normalice resultados y genere un informe accionable.

Temas

#	Tema	Por qué importa
1	Revisión de código segura (manual)	Encuentra fallos de lógica que las herramientas no ven
2	SAST (análisis estático)	Detecta patrones inseguros sin ejecutar el código
3	DAST (análisis dinámico)	Prueba la app en ejecución como lo haría un atacante
4	IAST (instrumentado)	Une visibilidad interna y ejecución real
5	SCA (composición/dependencias)	La mayoría del código es de terceros: CVE y licencias
6	Integración en CI (shift-left)	Detecta temprano y barato, en cada commit/PR
7	Triaje de hallazgos	Prioriza lo explotable y elimina el ruido que mata la adopción
8	Automatización (scripting, SARIF)	Pega las herramientas y normaliza la salida

Modelo mental y caso de decisión

Automatizar seguridad exige entradas confiables, idempotencia, límites, logging y rollback; un script con privilegios es parte de la superficie.



Caso razonado. Una remediación masiva corrige el síntoma y rompe servicio; el modo simulación y rollback limitan daño.

Glosario operativo complementario

Término	Definición
Evidencia	Información cuya procedencia y relación con un criterio pueden revisarse.
Supuesto	Condición declarada de la que depende una conclusión.
Riesgo residual	Exposición que permanece después del tratamiento.

Criterio de dominio

El alumno demuestra dominio cuando explica el diagrama, resuelve el caso con evidencia, declara límites y puede defender por qué su decisión cambia si cambia un supuesto.

Definiciones y características

- **Revisión de código segura:** lectura estructurada del código buscando defectos de seguridad (validación, authz, manejo de secretos, criptografía). Característica clave: capta **fallos de lógica de negocio** que ninguna herramienta automática entiende.

- **SAST (Static Application Security Testing):** analiza el **código fuente o binario sin ejecutarlo**, buscando patrones inseguros y flujos de datos peligrosos (taint). Característica clave: temprano y con cobertura amplia, pero propenso a **falsos positivos**.
- **DAST (Dynamic Application Security Testing):** prueba la aplicación **en ejecución** desde fuera, sin ver el código (caja negra). Característica clave: baja tasa de falsos positivos en lo que encuentra, pero **no ve** rutas no ejercitadas.
- **IAST (Interactive AST):** **instrumenta** la app en ejecución (agente) para observar el código desde dentro mientras corren las pruebas. Característica clave: combina precisión de contexto con validación en tiempo de ejecución.
- **SCA (Software Composition Analysis):** identifica **dependencias de terceros** y las cruza con bases de vulnerabilidades (CVE/GHSA) y licencias. Característica clave: cubre el **riesgo de la cadena de suministro**, hoy el mayor volumen de código.
- **Shift-left:** desplazar las pruebas de seguridad lo más temprano posible en el ciclo (IDE, commit, PR). Característica clave: cuanto antes se detecta, **más barato** es corregir.
- **Triage de hallazgos:** proceso de clasificar cada hallazgo (verdadero/falso positivo, severidad, explotabilidad, acción). Característica clave: sin triaje, el ruido entierra lo importante y el equipo deja de mirar los informes.
- **SARIF:** formato estándar (OASIS) de resultados de análisis estático, consumido por GitHub Code Scanning y otras plataformas. Característica clave: **normaliza** salidas de herramientas distintas para agregarlas y deduplicarlas.

Herramientas y preparación

Entorno **de laboratorio propio** con una app deliberadamente vulnerable (para practicar detección, no ataque a terceros):

- **App objetivo:** OWASP **Juice Shop** o **WebGoat**, o un repo de ejemplo con vulnerabilidades conocidas (todo en tu máquina/laboratorio).
- **SAST:** **Semgrep** (reglas open source `p/owasp-top-ten`), **Bandit** (Python) o **CodeQL** (GitHub).
- **SCA:** **OWASP Dependency-Check**, **Trivy** (`trivy fs`) o `pip-audit / npm audit`.
- **DAST:** **OWASP ZAP** (modo baseline/automation framework) contra la app en tu laboratorio.
- **Secretos:** **gitleaks** o **detect-secrets** para credenciales filtradas en el histórico.
- **CI:** repositorio con **GitHub Actions** (o GitLab CI) para orquestar los análisis en cada PR.
- **Scripting:** Python o Bash para lanzar herramientas, parsear **JSON/SARIF** y generar el informe.

Nota ética: todo el análisis dinámico se hace contra **aplicaciones propias o autorizadas** en tu laboratorio. El propósito es **defensivo** —encontrar y remediar—; no escanees sistemas de terceros sin permiso explícito por escrito.

Laboratorio guiado — Revisión de código + SAST/SCA en CI con triaje

 **Laboratorio ejecutable del programa:** `devsecops-pipeline` — su script `auditar.sh` orquesta las ocho capas y distingue siempre *sin hallazgos* de *no ejecutada*, y `priorizar.py` implementa la priorización KEV → EPSS → CVSS contra las APIs reales.

Ejercicio aplicado: revisas código manualmente, montas análisis automatizado en CI, triras los hallazgos y automatizas el informe.

1. **Revisión manual.** Toma un endpoint vulnerable (p. ej. una consulta SQL construida por concatenación) y, con el checklist de **OWASP Code Review / ASVS**, identifica el fallo, su categoría (inyección) y la corrección (consulta parametrizada). Documenta el antes/después.
2. **Ejecuta SAST local.** Corre `semgrep --config p/owasp-top-ten` . (o Bandit) y localiza el mismo hallazgo. Observa además algún **falso positivo** y anótalo.
3. **Ejecuta SCA.** Corre `trivy fs` . o `dependency-check` sobre las dependencias; identifica una librería con CVE y su versión corregida.
4. **Ejecuta DAST en laboratorio.** Lanza `zap-baseline` contra tu instancia local de Juice Shop y compara: ¿qué encontró DAST que SAST no, y viceversa? Explica por qué (dentro vs fuera).
5. **Integra en CI.** Crea un workflow de **GitHub Actions** que, en cada PR, ejecute SAST (Semgrep con salida **SARIF**) y SCA, suba los resultados a Code Scanning y **falle el build** si hay hallazgos de severidad alta.
6. **Define el umbral (quality gate).** Configura qué severidad rompe el pipeline y qué queda como aviso; justifica el umbral para no bloquear al equipo con ruido.
7. **Tría los hallazgos.** Para cada resultado, decide: verdadero positivo (corregir), falso positivo (suprimir con comentario/ `nosem` /baseline) o riesgo aceptado (excepción con caducidad y responsable). Prioriza por severidad × explotabilidad.
8. **Corrige y verifica.** Aplica la remediación de SQLi y de la dependencia vulnerable; reejecuta el pipeline y confirma que el hallazgo desaparece (no que se suprimió sin arreglar).
9. **Automatiza el informe.** Escribe un script (Python/Bash) que lea los JSON/SARIF de las herramientas, **deduplique**, cuente por severidad y emita un resumen accionable (Markdown) para el PR.
10. **Mide la mejora.** Compara hallazgos antes/después y el tiempo de detección; documenta la reducción de deuda de seguridad.

Entregable: informe de revisión manual (antes/después), workflow de CI con SAST+SCA en SARIF y quality gate, tabla de triaje con decisiones justificadas, evidencia de corrección verificada y el script de automatización del informe.

Ejercicios

1. Revisa un fragmento con concatenación SQL y reescríbelo con consulta parametrizada; nombra la regla ASVS aplicable.
2. Construye una tabla que compare SAST/DAST/IAST/SCA por: qué ven, cuándo corren, falsos positivos y falsos negativos típicos.
3. Escribe un workflow de GitHub Actions que ejecute Semgrep y falle si hay hallazgos `ERROR` .
4. Tría cinco hallazgos dados: marca verdadero/falso positivo, severidad y acción, con justificación.
5. Suprime un falso positivo de forma trazable (comentario `nosemgrep` o baseline) y explica por qué no es "ignorar".
6. Escribe un script que parsee un SARIF y cuente hallazgos por severidad y por regla.

Reto verificable

Reto: entrega un pipeline de análisis de código que detecte, triñe y verifique la corrección de vulnerabilidades reales en una app de laboratorio propia.

Criterio de aceptación:

- Hay una **revisión de código manual** documentada (antes/después) guiada por OWASP Code Review/ASVS, con al menos un fallo de lógica corregido.
- El pipeline de **CI ejecuta SAST y SCA** en cada PR, emite **SARIF** y **falla el build** por encima de un umbral de severidad justificado.
- Cada hallazgo pasa por **triaje** con decisión explícita (corregir / falso positivo suprimido de forma trazable / riesgo aceptado con caducidad).
- Al menos una **inyección** y una **dependencia vulnerable** quedan **corregidas y verificadas** (el hallazgo desaparece por arreglo, no por supresión).
- Existe un **script de automatización** que normaliza y deduplica resultados y produce un informe accionable.

⚠ Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
"SAST tira 400 hallazgos y nadie los mira"	Sin triaje ni umbral, el ruido mata la adopción. Prioriza por severidad×explotabilidad y ajusta reglas.
"DAST no encontró la SQLi que SAST sí vio"	DAST solo prueba lo que se ejercita. Cubre rutas con pruebas o combínalo con SAST/IAST.
"Suprimí el hallazgo para que pasara el build"	Suprimir ≠ corregir. Solo se suprime un falso positivo justificado; los reales se arreglan.
"El SCA marca un CVE en una dependencia que no uso"	Falso positivo por dependencia transitiva no alcanzable. Verifica el alcance real y documenta la excepción.
"Metí secretos en el repo y ya los borré del último commit"	Siguen en el histórico. Usa gitleaks sobre todo el historial y rota la credencial expuesta.
"El pipeline tarda 40 min y bloquea todo"	Análisis pesado en cada push. Usa escaneo incremental/diff-aware y reserva el completo para nightly.

? Preguntas frecuentes

? **¿SAST o DAST? ¿Cuál elijo?** Ambos: son complementarios. SAST ve el **código** (temprano, amplio, con ruido) y encuentra fallos en rutas no ejecutadas; DAST ve la **app corriendo** (preciso, tardío) y valida lo explotable de verdad. Un programa maduro usa SAST + SCA en cada PR y DAST periódico o en preproducción, más IAST donde se pueda instrumentar.

? **¿Por qué importa tanto el SCA si mi código está limpio?** Porque la mayor parte de una aplicación moderna es **código de terceros**. Una dependencia con un CVE crítico te vuelve vulnerable aunque tu código sea impecable. El SCA vigila esa cadena de suministro y te avisa de la versión corregida; sin él, tienes un punto ciego enorme.

? **¿Cómo evito que las herramientas frenen al equipo de desarrollo?** Con **shift-left bien calibrado**: reglas curadas, escaneo incremental sobre el diff, un quality gate que solo rompe por severidad alta y triaje ágil de falsos positivos. La seguridad que ralentiza sin criterio se acaba desactivando; la que da hallazgos precisos y rápidos se adopta.

? ¿Este contenido no es "ofensivo"? No: es defensivo. El objetivo es **encontrar y corregir** debilidades en tu propio software para reducir la superficie de ataque. Se practica sobre aplicaciones propias o de laboratorio (Juice Shop/WebGoat); no se ataca software de terceros. Es exactamente el trabajo de AppSec y de un pentester que reporta para remediar.

Referencias

- OWASP. *Code Review Guide* — owasp.org/www-project-code-review-guide.
- OWASP. *Application Security Verification Standard (ASVS)* — owasp.org/www-project-application-security-verification-standard.
- CompTIA. *PenTest+ (PT0-002/PT0-003) Exam Objectives* — dominio *Tools and Code Analysis*.
- Chapple, Stewart & Gibson. *(ISC)² CISSP Official Study Guide*, 9.^a ed., Sybex — *Software Development Security*.
- OASIS. *SARIF v2.1.0* — docs.oasis-open.org/sarif · Semgrep — semgrep.dev.

Material descargable

-  [Guía en PDF](#) — versión imprimible de esta clase.
-  [Presentación \(PPTX\)](#) — deck para proyectar en clase.

Clase anterior

[Clase 329 — Arquitectura de seguridad empresarial y Zero Trust](#)

Siguiente clase

[Clase 331 — IA generativa y LLMs en ciberseguridad: panorama, capacidades y límites](#)